

Construção de Sistemas Operacionais

Prof. Angelo Elias Dal Zotto

Escola Politécnica

PUCRS

Trabalho 1

Objetivos

- Expandir o conhecimento sobre construção e suporte de distribuições do sistema operacional Linux.
- Implementação e instalação de um servidor Web.
- Aprendizado sobre o conteúdo do diretório /proc.

Descrição

A implementação deste trabalho consiste na geração de uma distribuição Linux que possua um servidor web juntamente com uma aplicação escrita em C responsável por coletar informações do sistema e gerar uma página HTML dinamicamente. O servidor Web deverá servir essa página HTML, com o objetivo de fornecer informações básicas sobre o funcionamento do sistema (target/guest) a partir de um navegador na máquina host. Abaixo, segue a lista de informações que deverão ser apresentadas na página de maneira dinâmica:

- Versão do sistema e kernel;
- Uptime (em dias, horas, minutos e segundos);
- Tempo ocioso (em dias, horas, minutos e segundos);
- Data e hora do sistema;
- Modelo do processador, velocidade e número de núcleos;
- Carga do sistema;
- Capacidade ocupada do processador (em percentual);
- Quantidade de memória RAM total e usada (em megabytes);
- Operações sobre o sistema de entrada e saída (leituras e escritas);
- Sistemas de arquivos suportados pelo kernel;
- Dispositivos (caracter e bloco) e grupos;
- Dispositivos de rede;
- Lista de processos em execução (PID e nome).

Web Server

O webserver a ser utilizado é o busybox httpd. É necessário integrá-lo à imagem a partir do menu `make busybox-menuconfig`. Também é necessário criar um script que faz a inicialização automática do serviço com os argumentos corretos, mas para testar é possível ativar manualmente pelo QEMU com `httpd -f`. O servidor httpd deve servir a pasta `/var/www` e executar como um usuário não-privilegiado (ex: httpd), mas para testar é possível executar como `root`. A página inicial do servidor http (`index.html`) deve redirecionar automaticamente para a página da solução implementada que deve, obrigatoriamente, residir na pasta `/var/www/cgi-bin`. Para testar, é possível acessar diretamente a página CGI, por exemplo, através de `192.168.1.10/cgi-bin/monitor`.

Atualização dinâmica das informações

Observe que algumas das informações acima são dinâmicas (mudam constantemente), dessa forma, a cada vez que o usuário acessar a página, o servidor deverá apresentar as informações atualizadas. Uma técnica para criar programas executáveis no servidor que geram conteúdo web dinâmico em resposta a requisições HTTP é o CGI (Common Gateway Interface) em C, ou seja, a página requisitada é gerada dinamicamente por um programa escrito em C.

As informações enumeradas anteriormente deverão ser obtidas através do pseudo-filesystem `/proc` (abrindo arquivos com operações `open()` ou `fopen()`) e lendo essa informação diretamente (operações `read()` ou `fread()`) e não por meio de comandos do shell invocados a partir da aplicação. A aplicação é responsável por acessar os arquivos no diretório `/proc`, percorrer a estrutura de diretórios interna e filtrar o conteúdo necessário para geração do conteúdo HTML sem o uso de aplicações externas. Lembre-se, essa é uma aplicação C embarcada que não deve possuir dependências. A aplicação C deve ser copiada para a pasta `cgi-bin` dentro do diretório servido pelo httpd. Essa aplicação deve imprimir para o stdout a página HTML contendo os dados monitorados. Um exemplo básico é dado abaixo:

```
#include <stdio.h>

int main(void) {
    printf("Content-Type: text/html\r\n\r\n");
    printf("<html><body>");

    /* Corpo do HTML aqui */

    printf("</body></html>");
    return 0;
}
```

Obtendo data e hora

Para que a data e hora do sistema sejam disponibilizadas em `/proc/driver/rtc` será necessário habilitar o driver durante a configuração da distribuição.

Entrega

Este trabalho deverá ser realizado em grupos de 3 ou 4 integrantes (a ser indicado no Moodle) e apresentado no dia 10/04 (apresentação em torno de 5 minutos). Para a entrega, é esperado que apenas um dos integrantes envie pelo Moodle um arquivo compactado do projeto com o nome dos integrantes, contendo os scripts criados (arquivo `.config` e pasta `custom-scripts`, por exemplo) e os arquivos contendo o código fonte da aplicação implementada (pasta `apps`, por exemplo). Um exemplo de como compactar os arquivos necessários é dado no comando abaixo:

```
zip -r config.zip .config custom-scripts apps
```